BiNPU: A 33.0 MOP/s/LUT Binary Neural Network Inference Processor Showing 88.26% CIFAR10 Accuracy With 1.9 Mbit On-Chip Parameters in a 28-nm FPGA

Gil-Ho Kwak, *Student Member, IEEE*, and Tae-Hwan Kim, *Member, IEEE*

***Abstract*—An efficient processor to perform inference of binary neural networks (BNNs) is presented. The proposed processor, named BiNPU, is designed based on a unified architecture that can efficiently process BNN modules of various types, including those with group convolution and global average pooling, in a consistent output-parallel mechanism without resource overhead. Implemented in a 28 nm FPGA, BiNPU shows the resource efficiency as high as 33.0 MOP/s/LUT, 35.3% higher than the previous state-of-the-art processor that supports even fewer module types. BiNPU performs the CIFAR10 classification task achieving 88.26% accuracy with 1.9 Mbit parameters entirely stored in on-chip memories. The BRAM usage for implementing the on-chip memories is rather smaller than those of the previous processors stored some of the parameters in off-chip memories.**

***Index Terms*—Accelerator, microarchitecture, binary neural networks, inference, field programmable gate array.**

I. Introduction

BINARY neural networks (BNNs) are neural network models that leverage the compression technique [1] to quantize each parameter to a single bit without severely sacrificing inference quality. The binarization significantly reduces the memory footprint required for parameter storage, enabling the storage of most, if not all, parameters in on-chip memory. As a result, BNN inference might be implemented with minimal reliance on off-chip memory [2], [3]. This is beneficial to curtail the overall system cost as well as lower the power consumption by reducing off-chip memory accesses. BNNs are being therefore increasingly employed to implement energy-efficient inference processors for low-cost devices [4], [5], [6], [7].

Despite the benefits owing to binarization, implementing an efficient BNN inference processor remains significantly challenging in practice. By reducing the memory footprint to enable on-chip memory-based implementation, inference speed is not primarily limited by off-chip memory accesses, but rather by the available silicon resource. The conventional spatial architectures for full-precision models [8], developed by focusing on minimizing off-chip memory accesses, may therefore not be efficient to implement a BNN inference processor. Furthermore, even though the memory footprint is reduced by binarization, it can still impose a burden on on-chip memory for practical models (e.g., about 14M parameters for the CIFAR10 [9] classification model in [3], [6], [10]). There are techniques [11], [12] to reduce parameters for a smaller memory footprint, but an efficient architecture is needed to support these techniques.

There are several previous studies on efficient BNN inference processors. Batch normalization and sign activation were merged into a thresholding operation [4], [6], [13]. An output-stationary architecture developed based on the threshold- and pooling-based operation skipping techniques was presented [3]. Popcount was approximated by employing majority operations [14]. Binary multiplications were implemented by NAND operations [15]. An out-of-order architecture was presented with the operation skipping techniques assisted with regularization [6]. An efficient BNN inference processor was developed envisioning MCU integration [4]. A BNN inference processor with dual cooperative cores was presented [16]. An all-digital BNN inference processor was presented with the wide dot product operations and optimized memory access patterns [5]. An inference processor with low-precision input images and power-of-two parameters were developed [17].

Apart from the studies on the efficient BNN inference processors, some previous studies attempted to develop neural network models with reduced parameters. In MobileNet [11] and its successors, some of the general convolutions (CONVs) were replaced by the depthwise convolutions (DWCONVs) with fewer parameters. In ShuffleNet [12], DWCONVs were generalized to GCONVs. In several modern models [11], [12], the fully-connected layers are replaced by parameter-less GAP layers. Even though there are few of the previous architectures [18], [19] that supported some of the techniques above, it is not efficient to apply them straightforwardly to perform BNN inference. This is because they were developed

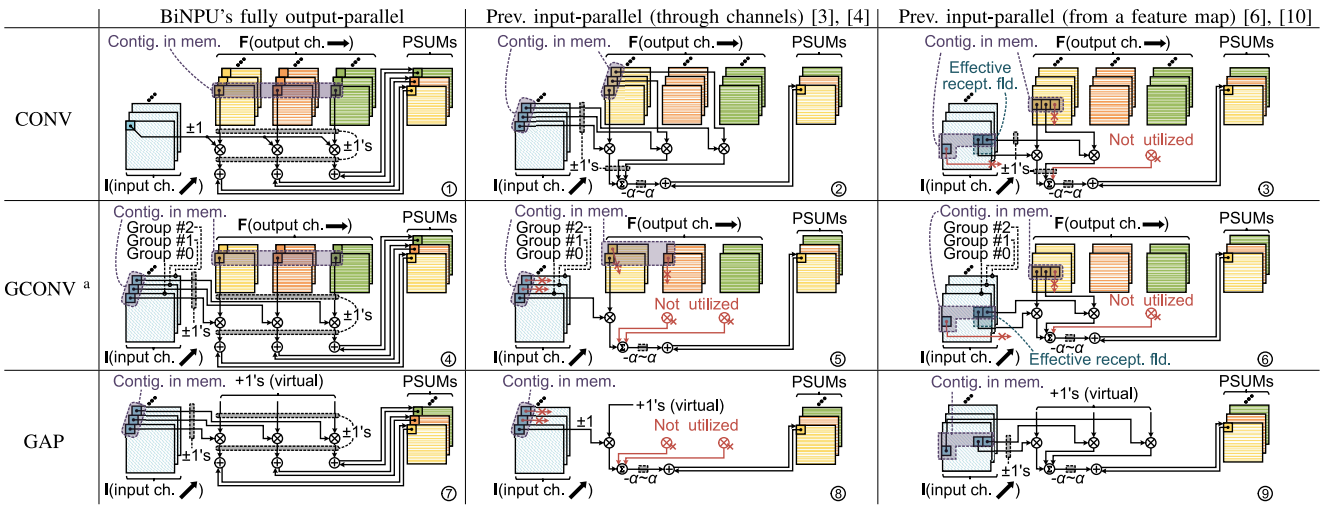
Manuscript received 1 April 2024; revised 22 July 2024; accepted 6 August 2024. Date of publication 8 August 2024; date of current version 30 October 2024. This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) Grant funded by the Korea Government (MSIT, The Basic Research Lab for Intelligent Semiconductor Working for the Multi-Band Smart Radar) under Grant 2017-0-00528. This brief was recommended by Associate Editor V. Gaudet. *(Corresponding author: Tae-Hwan Kim.)*

The authors are with the School of Electronics and Information Engineering, Korea Aerospace University, Goyang 10540, Gyeonggi, Republic of Korea (e-mail: taehwan.kim@kau.kr).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSII.2024.3440965.

Digital Object Identifier 10.1109/TCSII.2024.3440965

TABLE I
PROCESSING MECHANISMS, ILLUSTRATED FOR $\alpha = 3$ AND $c' = 3$



[a] The figures here illustrate the processing mechanisms for the GCONV module with the group size of 2 (i.e. $G = 2$), where the feature maps through consecutive channels belong to different groups. The input-parallel mechanism with the input elements from a feature map is illustrated for a possible situation with under-utilization of the compute units

to perform the inference of full-precision models whose compute structures are quite different from those of the BNNs. Furthermore, these techniques necessitate extra hardware units, which is undesirable for the systems with limited resources.

This brief proposes the design and implementation of an efficient BNN inference processor, named BiNPU.[1] The contributions are summarized as follows: leftmargin=0.5cm

- A unified architecture is proposed to efficiently process various BNN modules in a consistent output-parallel mechanism without resource overhead. Such modules include those with the group convolution (GCONV) and global average pooling (GAP), which are known to be beneficial to reduce parameters.
- The resource efficiency is as high as 33 MOP/s/LUT in a 28 nm FPGA, while various module types are supported. 88.26% CIFAR10 accuracy is achieved based on the BNN model designed by employing the various modules efficiently supported, so that entire parameters can be reduced to be fit into 1.9 Mbit on-chip memories.

The rest of this brief is organized as follows. Section II describes the inference flow based on the BNN models with the various modules considered in this brief. Section III presents BiNPU in detail. Section IV shows the implementation results and evaluates them. Finally, Section V draws the conclusion.

II. INFERENCE FLOW BASED ON BNN MODELS WITH CONV, GCONV AND GAP MODULES

This brief considers the XNOR-Net-based BNN models [20] with the parameter reduction techniques applied. Fig. 1 shows the overall inference flow based on these BNN models. The inference flow is composed of several modules connected in series, where the type of each module is specified by

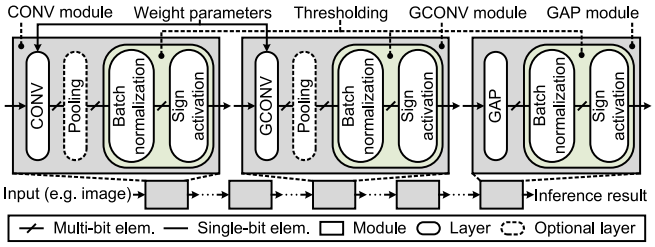


Fig. 1. Overall inference flow based on the BNN models considered in this brief and the three different types of modules that may be part of the flow.

the first layer therein. Each module has a *binary-input and binary-output* compute structure; that is, the feature maps and weights are composed of single-bit elements. The parameter reduction techniques, GCONV and GAP,[2] are applied in a way that preserves such a modular structure with binary inputs and binary outputs: each of the GCONV and GAP layers is followed by a batch normalization and sign-activation layers, so that the feature elements in between modules are binarized. This is different from the applications of these techniques to full-precision models without considering the binarization [11], [12]. By employing a GCONV module instead of a CONV module, the weight parameters can be reduced by a factor of G/c, where G and c denote the group and channel sizes, respectively. A GAP module involves no weight parameters.

III. PROPOSED PROCESSOR: BINPU

BiNPU processes BNN modules in a *fully output-parallel* mechanism. Table I contrasts this mechanism to those of the previous processors by illustrating the processing steps for the first layers of different module types with $\alpha = 3$ and $c' = 3$, where α and c' denote the parallelism factor and output

[1] BiNPU stands for a BNN inference processor based on a unified architecture.

[2] The details of GCONV and GAP are not described here for brevity and interested readers are referred to the previous literature [11], [12].

channel size, respectively.[3] In the table, **I** and **F** denote the input and weight binary tensors, respectively; $\otimes$ is an XNOR operation; PSUM stands for a partial sum. In BiNPU, α output elements are always computed in parallel for each processing step, so called output-parallel; whereas in the previous processors, multiple ($\leq \alpha$) input elements are used to compute one output element, so called input-parallel. In the mechanisms, we assume that the input elements are loaded from a contiguous memory region, without considering the gathering operations with non-unity strides that may be variable to module types. The reason for this is that such gathering operations would be so complicated as to be inappropriate for a low-complexity BNN inference processor. In addition, the weight parameters are stored in memory in an order that allows them to be loaded from a contiguous region, where this order differs for each processing mechanism.

The BiNPU's output-parallel mechanism is efficient in supporting various module types. The input-parallel mechanism of the previous processors is not efficient if not all of the α elements can be used for the computation and thus some compute units are not utilized. In the input-parallel mechanism, if the input elements are through channels and not in a single group for processing a GCONV layer (⑤ in Table I), the elements in one group cannot be used to compute the output elements to be computed with those in the other groups. If the input elements are from a feature map within a channel for processing a CONV or a GCONV layer (③, ⑥ in Table I), the elements outside a receptive field cannot be used.[4] The problem is that such under-utilization problem may occur depending on the module type, as illustrated in Table I. In contrast, in the BiNPU's output-parallel mechanism, α output elements through channels are always involved in the computation if the number of the output channels is not less than α, and this is viable regardless of the module type.

A unified architecture is designed to process modules of various types based on the consistent output-parallel mechanism without resource overhead. Fig. 2 describes the pipeline of the unified architecture. Each processing step illustrated in Table I is fulfilled per cycle through the pipeline. In Stage 1, the addresses for accessing the feature map and parameter memories are generated by concatenating the coordinates computed with the counter values corresponding to the induction variables in the compute loops for processing a module. In Stage 2, the vectors are loaded from the memories at the generated addresses and the inputs of the subsequent XNOR operations are selected according to the module type. In Stage 3, the XNOR operations are performed with the vectors and the PSUMs are *incremented* by the results of the XNOR operations, where the PSUMs are first initialized with the

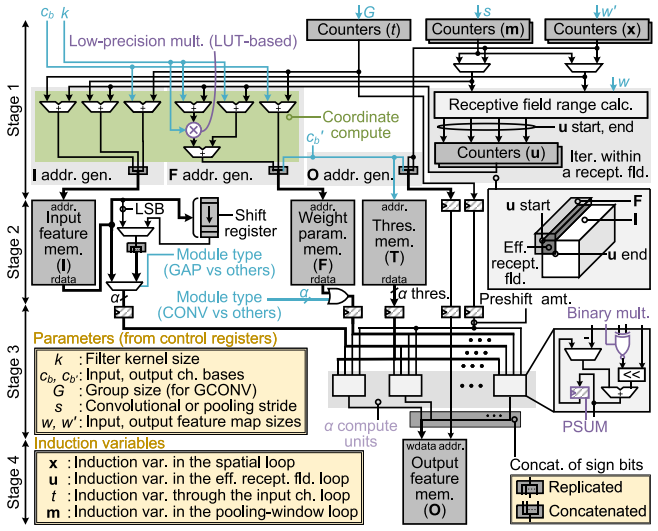


Fig. 2. Pipeline of the proposed architecture for module processing in BiNPU.

thresholds. The results of the XNOR operations can be prescaled by shifting for handling a non-binary input feature map (generally for the first module in a model), where a bitwise shift operation is denoted by $\ll$. In Stage 4, the resulting output elements, obtained by picking the sign bits of the cumulative sums (sign activation in Fig. 1), are stored to the memory for the output feature map. The resource overhead to support various module types is negligible; the processing of modules of different types share the same data path, except for the multiplexing logic in Stage 2, which are not considerable in terms of the resource usage.

The inference speed of the proposed unified architecture is scalable to the parallelism factor because the logic delay is not affected by the parallelism factor. The architectures of the previous processors [3], [6], [10], [17], [18] usually included multi-level adder trees to perform the multi-operand additions involved in computing the PSUMs. If these previous architectures were materialized so as to achieve a higher inference speed by processing more operands at once (i.e., with a higher parallelism factor), the logic delays of the adder trees would be lengthened with more levels, possibly failing in increasing an inference speed. In the proposed architecture, however, there are no adder trees, but rather multiple adders, working in parallel to increment the PSUMs, in which the logic delay is not affected by the parallelism factor. Hence, a higher inference speed with a higher parallelism factor can be achieved without revising the architecture.

BiNPU is developed based on the proposed unified architecture and integrated into an inference system to verify its functionality. Fig. 3 shows the inference system, where the MCU provides test images to the inference processor and verifies if the results from the inference processor are correct. BiNPU incorporates one module processing unit (MPU) and a few of the on-chip memories that store feature maps and parameters. The MPU is designed based on the proposed unified architecture (with $\alpha = 128$), to be versatile in processing various modules. This allows the MPU to be utilized in a time-multiplexed way, processing the modules

[3]The mechanisms developed for BNN inference quite differ from those of the spatial architectures [8] developed for full-precision models (e.g., input and output stationary). The latter ones were developed with the approaches for data reuse by formulating dataflows between compute units. Such approaches may not be as effective for BNN inference, where the data amount is not so large; rather increase complexity. It is noteworthy that no dataflows are between compute units in the mechanisms (in Table I) for BNN inference.

[4]Even if the input-parallel mechanisms are extended to process multiple output elements simultaneously, they are not efficient, either, when the inputs are underutilized.

TABLE II
BNN MODELS DESIGNED TO VERIFY THE BINPU'S FUNCTIONALITY

Model structure [a]	Param.	CIFAR10	SVHN
$\mathbb{C}_{c=3}\rightarrow\mathbb{C}_{c=2^7,s=2}\rightarrow\mathbb{C}_{c=2^7}\rightarrow\mathbb{C}_{c=2^8,s=2}\rightarrow\mathbb{C}_{c=2^8}\rightarrow\mathbb{C}_{c=2^9,s=2}\rightarrow\mathbb{C}_{c=2^{13},k=1}\rightarrow\mathbb{C}_{c=2^{10},k=1}\rightarrow\mathbb{C}_{c=2^{10},k=1}$	14.02 Mb	88.91%	94.42%
$\mathbb{C}_{c=3}\rightarrow\mathbb{C}_{c=2^7,s=2}\rightarrow\mathbb{C}_{c=2^8,s=2}\rightarrow\mathbb{C}_{c=2^8,s=2}\rightarrow\mathbb{C}_{c=2^{12},k=1}\rightarrow\mathbb{C}_{c=2^7,k=1}$	2.00 Mb	85.63%	91.96%
$\mathbb{C}_{c=3}\rightarrow\mathbb{C}_{c=2^7,s=2}\rightarrow\mathbb{C}_{c=2^7}\rightarrow\mathbb{C}_{c=2^7,s=2}\rightarrow\mathbb{C}_{c=2^8}\rightarrow\mathbb{C}_{c=2^8,s'=2}\rightarrow\mathbb{C}_{c=2^8,k=1}\rightarrow\mathbb{C}_{c=2^8,k=1}\rightarrow\mathbb{G}_{c=2^8,G=1}\rightarrow\mathbb{G}_{c=2^8,G=2}\rightarrow\mathbb{C}_{c=2^7,k=1}\rightarrow\mathbb{A}_{c=2^7}\rightarrow\mathbb{C}_{c=2^7,k=1}$	1.93 Mb	88.26%	93.82%
$\mathbb{C}_{c=3}\rightarrow\mathbb{C}_{c=2^7,s=2}\rightarrow\mathbb{C}_{c=2^7}\rightarrow\mathbb{C}_{c=2^7,s=2}\rightarrow\mathbb{C}_{c=2^7}\rightarrow\mathbb{C}_{c=2^7,s'=2}\rightarrow\mathbb{C}_{c=2^8,k=1}\rightarrow\mathbb{C}_{c=2^8,k=1}\rightarrow\mathbb{G}_{c=2^8,G=1}\rightarrow\mathbb{G}_{c=2^8,G=2}\rightarrow\mathbb{C}_{c=2^7,k=1}\rightarrow\mathbb{A}_{c=2^7}\rightarrow\mathbb{C}_{c=2^7,k=1}$	1.04 Mb	86.28%	93.42%

[a] $\mathbb{C}$, $\mathbb{G}$, and $\mathbb{A}$ represent CONV, GCONV, and GAP modules, respectively. c denotes the input channel size. k denotes the filter kernel size, which is 3 if not specified for a convolutional module. $\mathbb{C}$ with $k = 1$ is equivalent to a conventional fully-connected module. s denotes the pooling or convolutional stride, where the latter one is represented with a prime symbol.

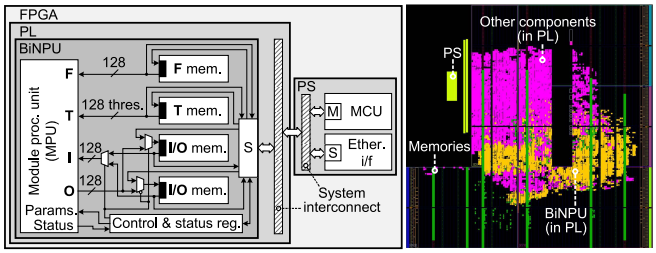


Fig. 3. Inference system to verify the functionality and floorplan in the device, where M and S stand for master and slave interfaces, respectively.

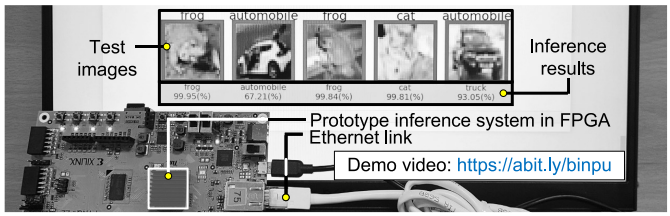


Fig. 4. Demonstration environment.

that make up a model one after the other, where their types may be different to each other. This leads to be more resource efficient compared to the previous processors [18] based on the heterogeneous architectures, in which each MPU is dedicated to process the modules of one type. It should be remarked that the inference speed and resource usage might be scaled by the parallelism factor; in addition, more MPUs might be incorporated to increase overall processing throughput.

IV. RESULTS AND EVALUATION

The BiNPU's functionality is verified for the practical inference tasks based on the models in Table II.[5] The first two models in the table use only CONV modules, with the first model corresponding to the conventional one in [3], [6], [10]; the other models incorporate GCONV and GAP modules alongside CONV modules, reducing parameters with minimal accuracy degradation. For example, the third model has 86.2% fewer parameters and less than 0.7% accuracy degradation on the CIFAR10 classification task compared to the first model. This degradation is less severe than that of the second model with a comparable number of parameters. Importantly, the BiNPU can process GCONV and GAP modules as well as CONV modules, unlike most previous processors, allowing it to perform accurate inference with reduced parameters.

The inference system integrating BiNPU is implemented in a 28 nm FPGA. The resource usage to implement the system is 15.1k LUTs and 14.5k FFs, of which only 5.1k LUTs and 4.8k FFs are used for BiNPU. The BRAM usage to implement the on-chip memories is 2274 kbits, of which 262 kbits, 1975 kbits, and 37 kbits are used for the feature maps, parameters, and thresholds, respectively. The sizes of the on-chip memories have been determined to be large enough to support the third model in Table II. The maximum operating frequency is 242 MHz. The power consumption of the inference system is 1.45 W, of which 96 mW is consumed by BiNPU, estimated under 25°C with a 1V supply and practical switching activity information of a default toggle rate (1/8) annotated. Fig. 4 demonstrates that the system is performing the task.

The implementation results in Table III show that BiNPU efficiently supports various module types. BiNPU is designed based on the unified architecture to process various modules without resource overhead, and it is advantageous to achieve a high speed with a low complexity due to no adder trees. As a result, compared to the previous state-of-the-art processor implemented in the same FPGA device, it supports more module types, rather showing a 35.3% higher resource efficiency. Here, resource efficiency is defined by the inference speed contributed by the unit resource usage, manifesting the architectural merit in terms of the resource usage [3], [13]. Though implemented in a different device, the processor in [3] shows a higher resource efficiency than BiNPU, but it supports only one type of module. Since the module types supported in addition to those in the previous processors are feasibly used to reduce parameters as shown in Table II, BiNPU performs inference with the parameters, that are all stored in on-chip memories, even though the sizes of the on-chip memories are not so large.

BiNPU does not involve any off-chip memory accesses for inference since entire parameters as well as feature elements are stored in the on-chip memories, reflected by the zero off-chip memory bandwidth in the table. Most of the other processors store the parameters in off-chip memories and copy them into on-chip memories for uses. Off-chip memory accesses consume an order of magnitude higher power than on-chip memory accesses [8]. Therefore, BiNPU would be much more energy efficient than the others requiring off-chip memory bandwidths, if the power consumption associated with off-chip memory accesses were taken into account. If the others were redesigned to store all parameters in on-chip memories, the sizes of the on-chip memories would be

[5] Since BNNs are useful in practice for small or medium scale tasks, where the achievable accuracies are close to those of the full-precision models, these kinds of tasks have been used here for verification purposes.

TABLE III
IMPLEMENTATION RESULTS OF THE LOW-PRECISION INFERENCE PROCESSORS IN FPGAS

	BiNPU	FPL-18 [10]	ELEC-21 [21]	TCAS2-21 [3]	TPDS-21 [6]	TCAS2-22 [18]	TCAS2-23 [17]
Supported module type(s)	GCONV, CONV, GAP	CONV	CONV	CONV	CONV	CONV, DWCONV	CONV
Precision (feat. / weight / PSUM)	1 / 1 / 16	1 / 1 / N. A.	1 / 1 / N. A.	1 / 1 / 16	1 / 1 / N. A.	4 / 4 / N. A.	4 / 6 / 25
Parameters (Mbit) [a]	1.926	14.022	14.022	14.022	14.022 [b]	N. A.	1.030
Accuracy (%) [a]	88.26	88.61	89.95	88.88	83.90 [b]	88.60	87.02
FPGA device	XC7Z020	XC7Z020	XCZU7EV	5CSXFC6D6	XC7Z045	XC7VX690T	XC7Z020
Logic res. usage (kLUT)	5.12	29.60	4.80	2.00	180.86 [c]	107.33	40.10
BRAM usage (kbit)	2,274	3,708	1,602	2,308	N. A.	13,700	1,548
Inference speed (GOP/s)	169.2	722.0	101.5 [d]	83.0	3,327.3	413.2	704.1
Off-chip mem. b.w. (Gbit/s) [e]	0	$\geq$7.3	$\geq$1.0	$\geq$0.3	$\geq$36.1	N. A. ($\gg$0)	0
Resource eff. (MOP/s/LUT)	33.0	24.4	21.1	41.5	18.4	3.8	17.5
Energy eff. (TOP/J) [f]	1.76 (0.12)	0.22 (N. A.)	0.25 (N. A.)	0.94 (N. A.)	N. A. (0.37)	0.07 (N. A.)	0.89 (N. A.)

[a] For the models designed for the CIFAR10 task; [b] Evaluated for the regularized model with a relaxed threshold; [c] Estimated with the CLB count by considering the LUT count per CLB; [d] Normalized to 28nm technology of the FPGAs for the other processors; [e] Estimated with the total inference time and model parameters stored in off-chip memories (0: no off-chip memory accesses); [f] Inside the parentheses is the efficiency of the system integrating the processor.

exorbitantly large that they could not be fit into such low-cost devices where BNNs are preferred to full-precision models for inference. The processor in [17] also performed inference with all the parameters stored in on-chip memories, but its accuracy is not as high even with a higher precision of such a model designed with smaller dimensions.

The BiNPU's architecture is not optimized for a certain specific model, but optimized to efficiently support various modules with a high utilization rate of the compute units. In fact, the compute units in BiNPU are fully utilized in processing every module in the models in Table II, except for the last modules whose output channel sizes are less than the number of the compute units. Therefore, such a high efficiency in Table III is maintained when running larger models, but possibly with a non-zero off-chip memory bandwidth.

V. CONCLUSION

An efficient BNN inference processor, BiNPU, was presented. BiNPU was designed to efficiently process the various BNN modules with the parameter reduction techniques, based on the unified architecture without resource overhead. Implemented in a 28 nm FPGA, BiNPU exhibited the resource efficiency as high as 33 MOP/s/LUT, successfully supporting the various module types with the parameter reduction techniques. To the best of our knowledge, this is the first study on the design and implementation of an BNN inference processor supporting the parameter reduction techniques. Future study might be followed to improve the processor for the complicated dataflows in advanced models.

ACKNOWLEDGMENT

The EDA tools were supported by IDEC.

REFERENCES

[1] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *Proc. Adv. Neur. Inf. Proc. Syst.*, 2016, pp. 4107–4115.

[2] B. Moons, D. Bankman, L. Yang, B. Murmann, and M. Verhelst, "BinarEye: An always-on energy-accuracy-scalable binary CNN processor with all memory on chip in 28nm CMOS," in *Proc. Custom Integr. Circuits Conf.*, 2018, pp. 1–4.

[3] T.-H. Kim and J. Shin, "A resource-efficient inference accelerator for binary convolutional neural networks," *IEEE Trans. Circuits Syst. II*, vol. 68, no. 1, pp. 451–455, Jan. 2021.

[4] F. Conti, P. D. Schiavone, and L. Benini, "XNOR neural engine: A hardware accelerator IP for 21.6-fJ/OP binary neural network inference," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2940–2951, Nov. 2018.

[5] P. C. Knag et al., "A 617-TOPS/W all-digital binary neural network accelerator in 10-nm FinFET CMOS," *IEEE J. Solid-State Circuits*, vol. 56, no. 4, pp. 1082–1092, Apr. 2020.

[6] T. Geng et al., "O3BNN-R: An out-of-order architecture for high-performance and regularized BNN inference," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 199–213, Jan. 2021.

[7] X. T. Nguyen, T. N. Nguyen, H.-J. Lee, and H. Kim, "An accurate weight binarization scheme for CNN object detectors with two scaling factors," *IEIE Trans. Smart Process. Comput.*, vol. 9, no. 6, pp. 497–503, Dec. 2020.

[8] Y.-H. Chen, J. Emer, and V. Sze, "Using dataflow to optimize energy efficiency of deep neural network accelerators," *IEEE Micro*, vol. 37, no. 3, pp. 12–21, Jun. 2017.

[9] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Rep. TR-2009, Apr. 2009.

[10] P. Guo, H. Ma, R. Chen, P. Li, S. Xie, and D. Wang, "FBNA: A fully binarized neural network accelerator," in *Proc. Int. Conf. Field-Prog. Logic & Appl.*, 2018, pp. 51–54.

[11] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[12] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6848–6856.

[13] Y. Umuroglu et al., "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. Int. Symp. Field.-Prog. Gate Arrays*, 2017, pp. 65–74.

[14] S. Rasoulinezhad, S. Fox, H. Zhou, L. Wang, D. Boland, and P. H. Leong, "MajorityNets: BNNs utilising approximate popcount for improved efficiency," in *Proc. Int. Conf. Field.-Prog. Tech.*, 2019, pp. 339–342.

[15] H. Kim, J. Sim, Y. Choi, and L.-S. Kim, "NAND-Net: Minimizing computational complexity of in-memory processing for binary neural networks," in *Proc. Int. Symp. High Perf. Comput. Archit.*, 2019, pp. 661–673.

[16] T. Kim, J. Shin, and K. Choi, "IOTA: A 1.7-TOP/J inference processor for binary convolutional neural networks with 4.7 K LUTs in a tiny FPGA," *Inst. Eng. Technol. Elec. Lett.*, vol. 56, no. 20, pp. 1041–1044, Sep. 2020.

[17] S. Sanjeet, B. D. Sahoo, and M. Fujita, "Energy-efficient FPGA implementation of power-of-2 weights-based convolutional neural networks with low bit-precision input images," *IEEE Trans. Circuits Syst. II*, vol. 70, no. 2, pp. 741–745, Feb. 2023.

[18] L. Xuan, K.-F. Un, C.-S. Lam, and R. P. Martins, "An FPGA-based energy-efficient reconfigurable depthwise separable convolution accelerator for image recognition," *IEEE Trans. Circuits Syst. II*, vol. 69, no. 10, pp. 4003–4007, Oct. 2022.

[19] L. Bai, Y. Zhao, and X. Huang, "A CNN accelerator on FPGA using depthwise separable convolution," *IEEE Trans. Circuits Syst. II*, vol. 65, no. 10, pp. 1415–1419, Oct. 2018.

[20] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: ImageNet classification using binary convolutional neural networks," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 525–542.

[21] J. Cho, Y. Jung, S. Lee, and Y. Jung, "Reconfigurable binary neural network accelerator with adaptive parallelism scheme," *Electronics*, vol. 10, no. 3, p. 230, Jan. 2021.